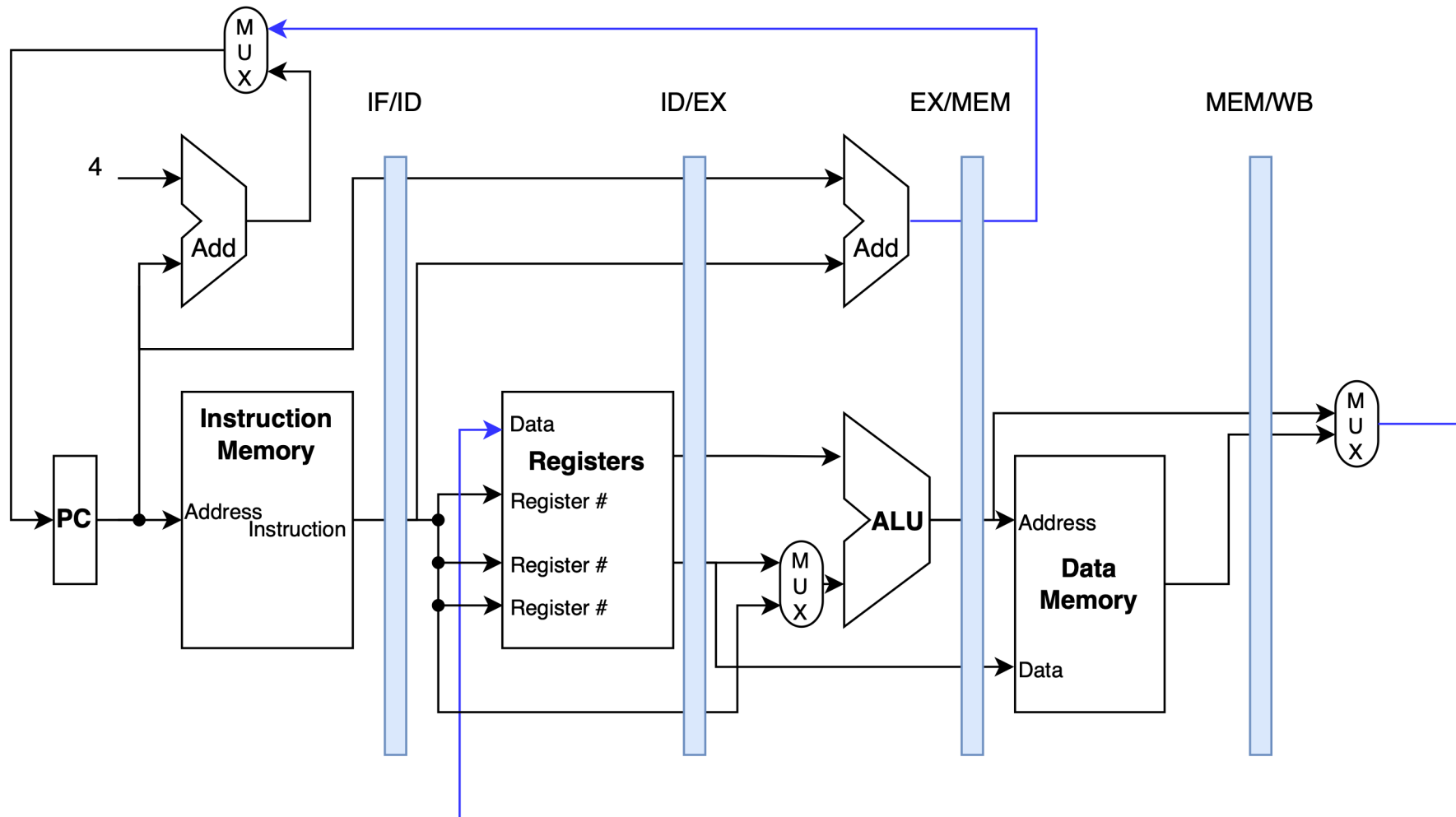


17. Pipelining (2)

Pipelined RISC-V



Pipelined RISC-V

- Blue rectangles represent registers
 - Varying bit widths
 - z-flag from ALU is held in the EX/MEM register
- Let's assume that the delay for each stage is as before:

IF	ID	EX	MEM	WB
200 ps	100 ps	200 ps	200 ps	100 ps

- Longest delay is 200 ps
- Therefore the minimum clock period is 200 ps
- Maximum clock speed is $1/200 \text{ ps} = 5 \text{ GHz}$

Speed Up

- We might have assumed that a 5-stage pipeline would be 5x faster than the single-cycle CPU
- Single-cycle clock frequency is 1.25GHz
- 5-stage pipeline frequency is 5GHz
 - 4x speed-up
 - Pipeline can only go as fast as its slowest part
- Note that the *latency* is now 1000 ps (was 800 ps) (time for one instruction to complete)

Throughput

- *Throughput* is one completed operation every 200 ps (5GHz)
 - $IPC = 1$, $CPI = 1$
- But this assumes that the pipeline is full all the time
 - In practice, this is not possible

Program Dependencies

- Example:

```
        lw      x1, 0(x7)      # instruction 1
        addi    x2, x0, 0      # instruction 2
loop:    addi    x2, x2, 1      # instruction 3
        addi    x9, x2, 3.     # instruction 4
        blt     x2, x1, loop   # instruction 5
        lw      x4, 0(x8)      # instruction 6
        addi    x3, x0, 3      # instruction 7
        add     x3, x3, x4      # instruction 8
        sub     x2, x3, x6      # instruction 9
        and     x4, x6, x2      # instruction 10
```

- Not particularly meaningful. Not well optimised

Control Dependencies

- Instruction 5 is a branch.
 - Instructions that follow the branch cannot execute until it is known that the branch is not taken
 - Therefore the control flow is dependent on the branch decision

Data Dependencies

- Flow dependency
 - instruction 8 writes to x3. instruction 9 reads from x3
- Anti-dependency
 - instruction 8 must read from x4 before instruction 10 writes to x4
- Output dependency
 - instructions 3 and 9 both write to x2. instruction 9 cannot complete until instruction 3 has completed, else instruction 5 reads the wrong value
- Anti-dependencies and output dependencies can be eliminated by the compiler – better choice of registers

Pipeline Hazards

- Program dependencies create *hazards* in the pipeline if the dependent instructions could both be in the pipeline at the same time
- The processor must detect these hazards and ensure that the instructions do not interfere
- *Control* or *branch hazards* can occur between instructions that have a control dependency
- *Data hazards* occur when instructions have a data dependency
- *Structural hazards* occur when two instructions try to use the same resource simultaneously

Branch Hazards

- Let's assume the classic 5 stage RISC pipeline for all these examples
- In the 5th cycle, the pipeline contains the following instructions:

IF	ID	EX	MEM	WB
<code>blt x2,x1,loop</code>	<code>addi x9,x2,3</code>	<code>addi x2,x2,1</code>	<code>addi x2,x0,0</code>	<code>lw x1,0(x7)</code>

- i.e. WB has instruction 1, MEM instruction 2, and so on.
- In the next cycle, instruction 6 would be loaded:

IF	ID	EX	MEM	WB
<code>lw x4,0(x8)</code>	<code>blt x2,x1,loop</code>	<code>addi x9,x2,3</code>	<code>addi x2,x2,1</code>	<code>addi x2,x0,0</code>

Branch Hazard (2)

- If the branch is taken, the program loops back to instruction 3 and instruction would be discarded
- One solution is to insert a `nop` or *bubble* into the pipeline instead:

IF	ID	EX	MEM	WB
<code>blt x2,x1,loop</code>	<code>addi x9,x2,3</code>	<code>addi x2,x2,1</code>	<code>addi x2,x0,0</code>	<code>lw x1,0(x7)</code>
nop	<code>blt x2,x1,loop</code>	<code>addi x9,x2,3</code>	<code>addi x2,x2,1</code>	<code>addi x2,x0,0</code>
<code>addi x2,x2,1</code>	nop	<code>blt x2,x1,loop</code>	<code>addi x9,x2,3</code>	<code>addi x2,x2,1</code>
<code>addi x9,x2,3</code>	<code>addi x2,x2,1</code>	nop	<code>blt x2,x1,loop</code>	<code>addi x9,x2,3</code>

Another View

- We can view the pipeline in a different way

	200	400	600	800	1000	1200	1400	1600
<code>addi x9,x2,3</code>	IF	ID	EX	MEM	WB			
<code>blt x2,x1,loop</code>		IF	ID	EX	MEM	WB		
nop			bubble	bubble	bubble	bubble	bubble	
<code>addi x2,x2,1</code>				IF	ID	EX	MEM	WB

- This shows where each instruction is in each cycle
 - This is useful for seeing data hazards, later

nop

- A nop – no operation – is an instruction that does nothing
- A compiler can insert a nop, but in the case of a branch hazard, it has to be done in the pipeline
- RISC-V has a pseudo instruction (or alias)
- `nop` in assembly is in fact

```
addi x0, x0, 0
```

- i.e. add 0 to the contents of x0 (0) and write back to x0
- Inserting a nop means that the IPC is now < 1 .

Stall on Branch

- This approach is known as *Stall on Branch*
 - Insert a `nop` or *bubble* into the pipeline until we know which way the branch will go
- Suppose that 10% of the instructions in a program are branch instructions. Suppose that every time we see a branch, we stall for one cycle.
 - This increases the CPI to 1.1
 - Reduces the IPC to 0.9

Branch Prediction

- A second approach is to try to predict each branch decision
- For example, always predict that a conditional branch will *not* be taken and the next instruction is loaded
 - If this is true, the pipeline is kept full
 - If not, execution stalls
- A more sophisticated approach tries to guess dynamically whether the branch will be taken and executes according to this assumption
 - Deep pipelines mean the pipeline has to restart from further back
 - *Speculative execution* – guessing the branch and executing – was exploited by the Spectre vulnerability (2018)

Delayed Decision

- A third approach is to reorder instructions
 - Fill the bubble with an instruction (from before the branch) that is not affected by the branch
 - Usually only one instruction is moved
 - If it's not possible to find an instruction, a nop is inserted
 - MIPS does this automatically – it's not visible to the programmer
 - Other CPUs leave this to the compiler/programmer, *but* a delayed branch instruction must be included in the ISA